NLP Lab - II

BoW in Python

Scikit Learn class to convert text data into a numerical representation suitable for machine learning models

```
from sklearn.feature_extraction.text import CountVectorizer
texts = ["I love machine learning", "Machine learning is amazing", "I love coding"]
vectorizer = CountVectorizer()
bow = vectorizer.fit_transform(texts) //Learn the vocabulary dictionary and return document-term matrix.
print(vectorizer.get_feature_names_out())
print(bow.toarray())
```

```
['amazing' 'coding' 'is' 'learning' 'love' 'machine']
[[0 0 0 1 1 1]
 [1 0 1 1 0 1]
 [0 1 0 0 1 0]]
```

BoW in Python

```
from sklearn.feature_extraction.text import CountVectorizer
>>> corpus = [ ‘This is the first document.’, 'This document is the second document.', 'And this is the third one.', 'Is this the first document?' ]
>>> vectorizer = CountVectorizer()
>>> X = vectorizer.fit_transform(corpus)
>>> vectorizer.get_feature_names_out()
array(['and', 'document', 'first', 'is', 'one', 'second', 'the', 'third', 'this’],….)

>>> print(X.toarray())
 [[0 1 1 1 0 0 1 0 1]
[0 2 0 1 0 1 1 0 1]
[1 0 0 1 1 0 1 1 1]
[0 1 1 1 0 0 1 0 1]]
```

Tuple (min,max)

```
vectorizer2 = CountVectorizer(analyzer='word', ngram_range=(2, 2))
>>> X2 = vectorizer2.fit_transform(corpus)
>>> vectorizer2.get_feature_names_out()
array(['and this', 'document is', 'first document', 'is the', 'is this', 'second
document', 'the first', 'the second', 'the third', 'third one', 'this document',
'this is', 'this the'], ...)

>>> print(X2.toarray())
[[0 0 1 1 0 0 1 0 0 0 0 1 0]
[0 1 0 1 0 1 0 1 0 0 1 0 0]
[1 0 0 1 0 0 0 0 1 1 0 1 0]
[0 0 1 0 1 0 1 0 0 0 0 0 1]]
```

For example, an ngram_range of (1, 1) means only unigrams, (1, 2) means unigrams and bigrams, and (2, 2) means only bigrams.

```
vectorizer2 = CountVectorizer(analyzer='word', ngram_range=(1, 2))
>>> X2 = vectorizer2.fit_transform(corpus)
>>> vectorizer2.get_feature_names_out()
['and' 'and this' 'document' 'document is' 'first' 'first document' 'is' 'is
the' 'is this' 'one' 'second' 'second document' 'the' 'the first' 'the second'
'the third' 'third' 'third one' 'this' 'this document' 'this is' 'this the']
>>> print(X2.toarray())
[[0 0 1 0 1 1 1 1 0 0 0 1 1 0 0 0 0 1 0 1 0]
 [0 0 2 1 0 0 1 1 0 0 1 1 1 0 1 0 0 0 1 1 0 0]
 [1 1 0 0 0 0 1 1 0 1 0 0 1 0 0 1 1 1 1 0 1 0]
 [0 0 1 0 1 1 1 0 1 0 0 0 1 1 0 0 0 0 1 0 0 1]]
```

TF-IDF in Python

```
from sklearn.feature_extraction.text import TfidfVectorizer
tfidf = TfidfVectorizer(). // # Step 1: Initialize the vectorizer (no vocabulary yet!)
X = tfidf.fit_transform(texts) //# Step 2: Learn vocabulary from `texts` + transform it
print(tfidf.get_feature_names_out()) //# Step 3: Now the vectorizer knows the vocabulary
print(X.toarray())
```

```
['amazing' 'coding' 'is' 'learning' 'love' 'machine’]
[[0. 0. 0. 0.57735027 0.57735027 0.57735027]
[0.5628291 0. 0.5628291 0.42804604 0. 0.42804604]
[0. 0.79596054 0. 0. 0.60534851 0. ]]
```

Similarity in Texts

$$J(A,B) = \frac{|A \cap B|}{|A \cup B|}$$

```
text1 = set("machine learning is fun".split())
text2 = set("learning about machine intelligence".split())
jaccard = len(text1 & text2) / len(text1 | text2)
print("Jaccard Similarity:", jaccard)
```

Jaccard Similarity: 0.3333333333333333

Cosine Similarity

$$\text{cosine similarity} = S_C(A, B) := \cos(\theta) = \frac{\mathbf{A} \cdot \mathbf{B}}{\|\mathbf{A}\|\|\mathbf{B}\|}$$

```
from sklearn.metrics.pairwise import cosine_similarity

tfidf_vec = TfidfVectorizer()

vecs = tfidf_vec.fit_transform(["machine learning is fun", "learning about machine intelligence"])

cos_sim = cosine_similarity(vecs[0:1], vecs[1:2])

print("Cosine Similarity:", cos_sim[0][0])
```

Cosine Similarity: 0.3360969272762575

Jaccard compares token sets;
Cosine compares vector angles (good for longer texts).

Use Cases:

- Useful when comparing **one document to another**
- You can also compare **one document to all others**

```
from sklearn.metrics.pairwise import cosine_similarity
from sklearn.feature_extraction.text import TfidfVectorizer
tfidf_vec = TfidfVectorizer()
vecs = tfidf_vec.fit_transform(["machine learning is fun", "learning about machine intelligence","helllo everyone","I"])
cos_sim = cosine_similarity(vecs[0:1], vecs)
print(cos_sim.shape)
print("Cosine Similarity:", cos_sim[0][0])
print(vecs)
```

```
(1, 4)
Cosine Similarity: 1.0
<Compressed Sparse Row sparse matrix of dtype 'float64'
        with 10 stored elements and shape (4, 8)>
  Coords        Values
  (0, 7)        0.43779123108611473
  (0, 6)        0.43779123108611473
  (0, 5)        0.5552826649411127
  (0, 2)        0.5552826649411127
  (1, 7)        0.43779123108611473
  (1, 6)        0.43779123108611473
  (1, 0)        0.5552826649411127
  (1, 4)        0.5552826649411127
  (2, 3)        0.7071067811865476
  (2, 1)        0.7071067811865476
```

```
print(vecs[0,2])
```

```
0.5552826649411127
```

```
print(tfidf_vec.get_feature_names_out())
```

```
['about' 'everyone' 'fun' 'helllo' 'intelligence' 'is' 'learning'
 'machine']
```

Sentiment Analysis

```
from textblob import TextBlob
review = "The service was excellent and the staff was friendly."
blob = TextBlob(review)
print("Polarity:", blob.sentiment.polarity)       // -1 to 1
print("Subjectivity:", blob.sentiment.subjectivity)     //0 to 1 0.0 is very objective and 1.0 is very subjective.
```

A subjective review is based on personal opinions, feelings, interpretations, and individual experiences.
An objective review is based on facts, data, and observable evidence without personal bias or emotions.

Word Cloud

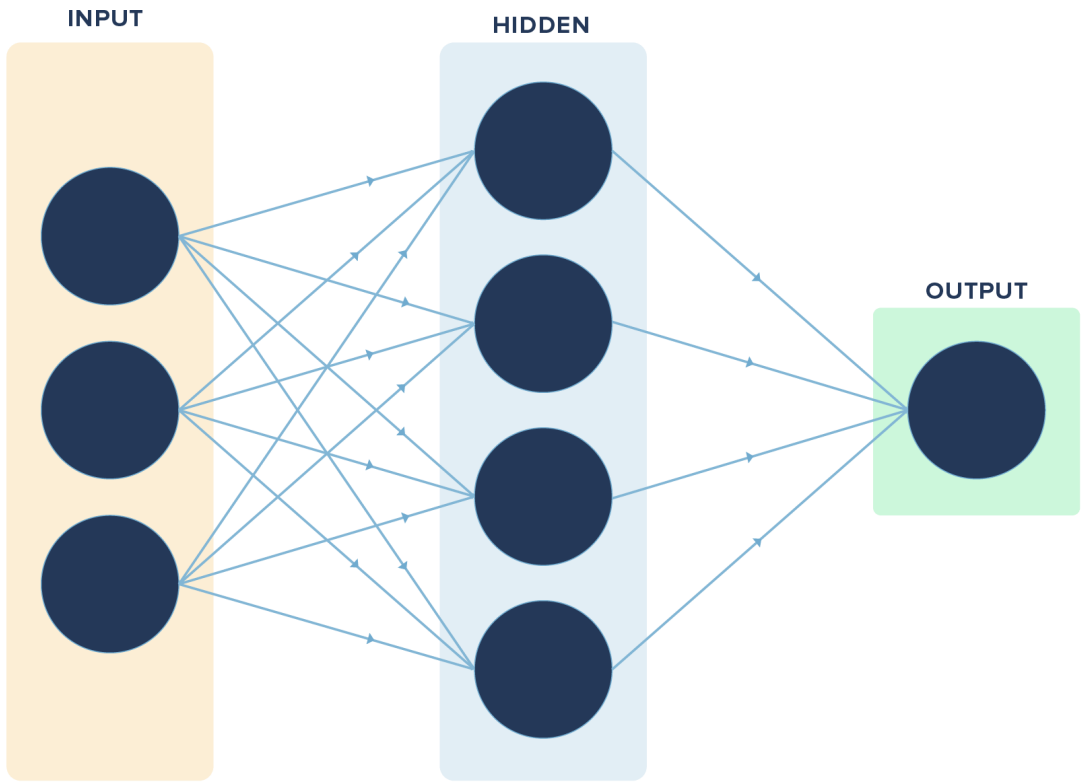
```
from wordcloud import WordCloud
import matplotlib.pyplot as plt
text = "Python is simple and powerful. I love Python programming!"
wordcloud = WordCloud().generate(text)
plt.imshow(wordcloud, interpolation='bilinear')
plt.axis('off')
plt.show()
```

	Machine Learning	Deep Learning
Definition	Subfield of AI that learns patterns from data	Subfield of ML that uses **neural networks** with multiple layers
Data Requirement	Works well with **small to medium** data	Needs **large volumes** of data
Feature Engineering	Requires **manual feature extraction**	Learns features **automatically**
Computation	Less computational power required	Needs **high GPU/TPU** power
Training Time	Usually **faster** to train	Can be **time-consuming**
Accuracy	Good accuracy for structured data	Often **higher accuracy**, especially with unstructured data
Examples	Linear Regression, SVM, Random Forest	CNNs, RNNs, LSTMs, Transformers
	Have **structured/tabular** data, want **quick results**, and have **less data**	Have **images, text, audio**, or **huge datasets** and want to **automate feature extraction**

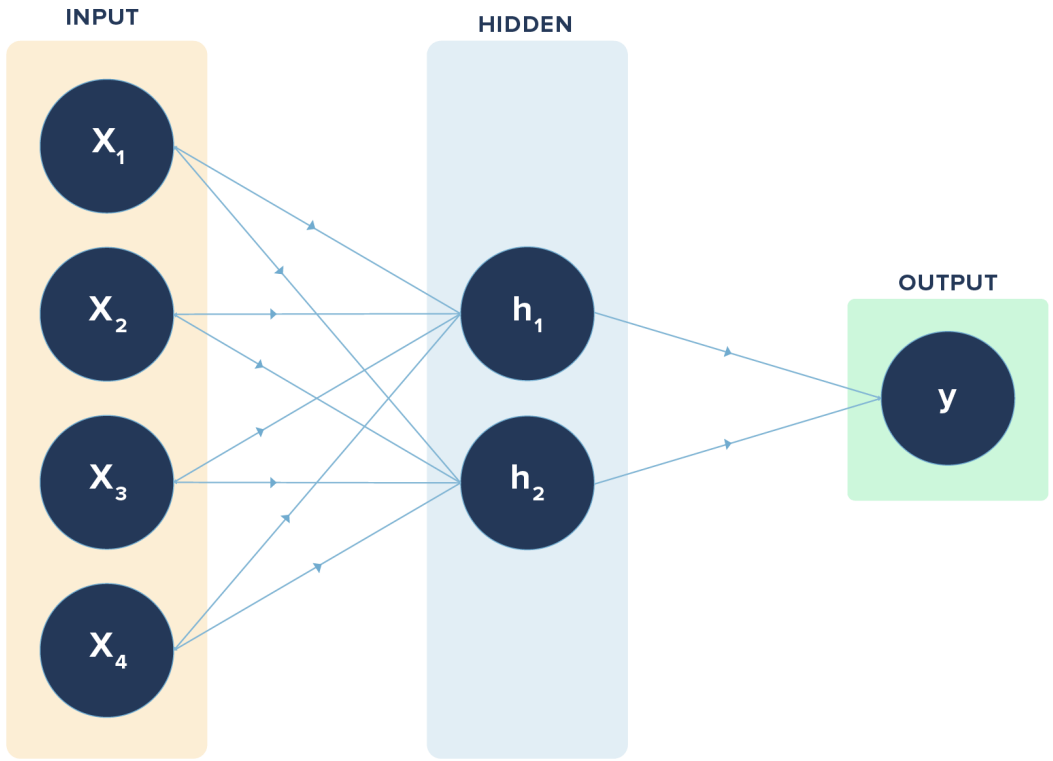
Neural Networks

- Machine learning model inspired by the way the human brain processes information.
- A **neural network** is a series of algorithms that attempts to recognize underlying relationships in a set of data through a process that mimics how the human brain operates.

Imagine neurons like people in an office. Each person (neuron) receives tasks (inputs), weighs their importance (weights), adds some experience (bias), makes a decision (activation), and passes it to the next person (next layer).s







- h1 = x1 * w1 + x2 * w1 + x3 * w1 + x4 * w1
  h2 = x1 * w2 + x2 * w2 + x3 * w2 + x4 * w2

- Neural Network - Complex Non-Linear models

Sigmoid Function $\sigma(z) = \frac{1}{1+e^{-z}}$

$z = \sum w_i x_i + b$

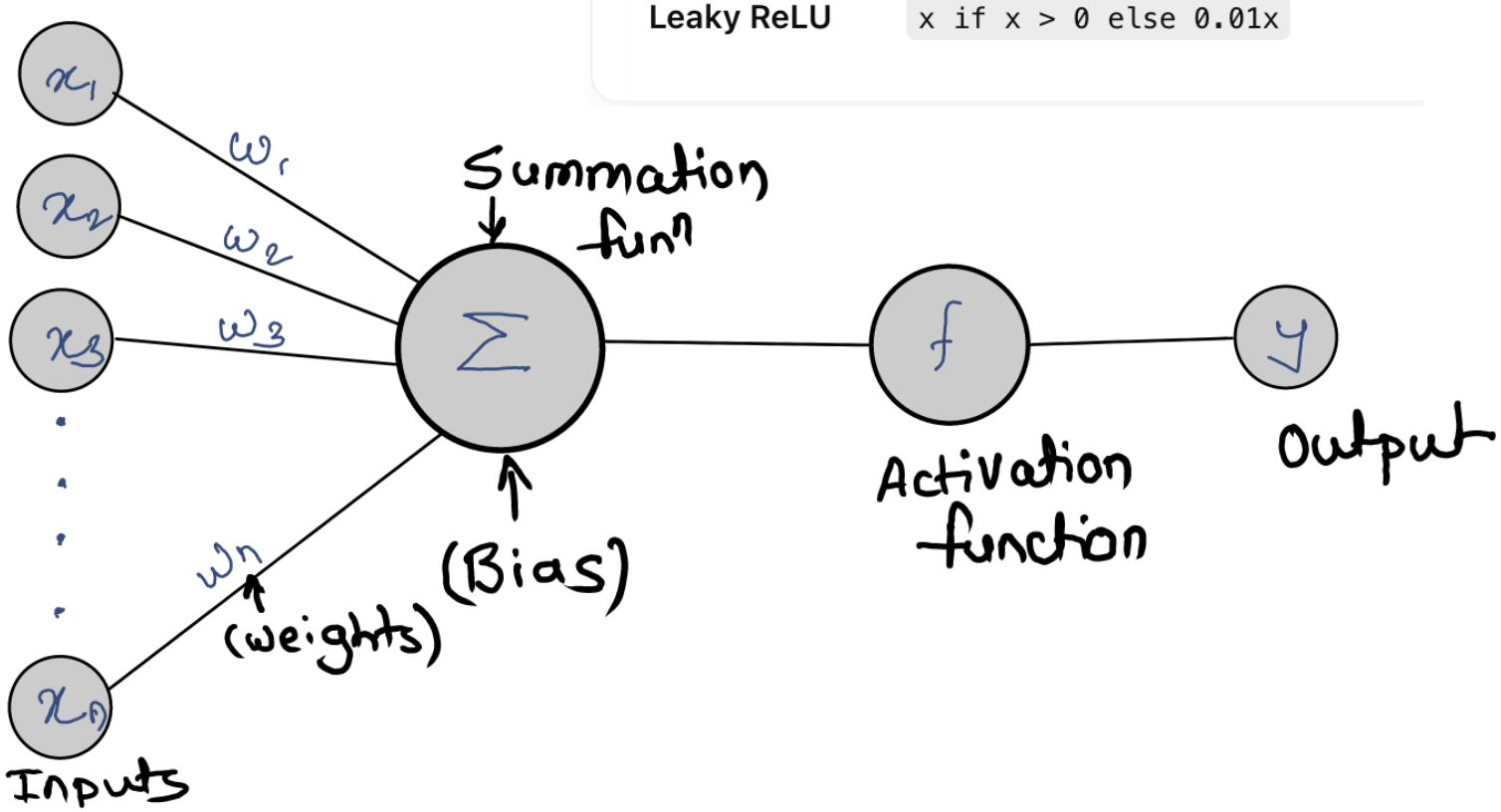
$= w_1x_1 + w_2x_2 + \ldots w_nx_n + b$

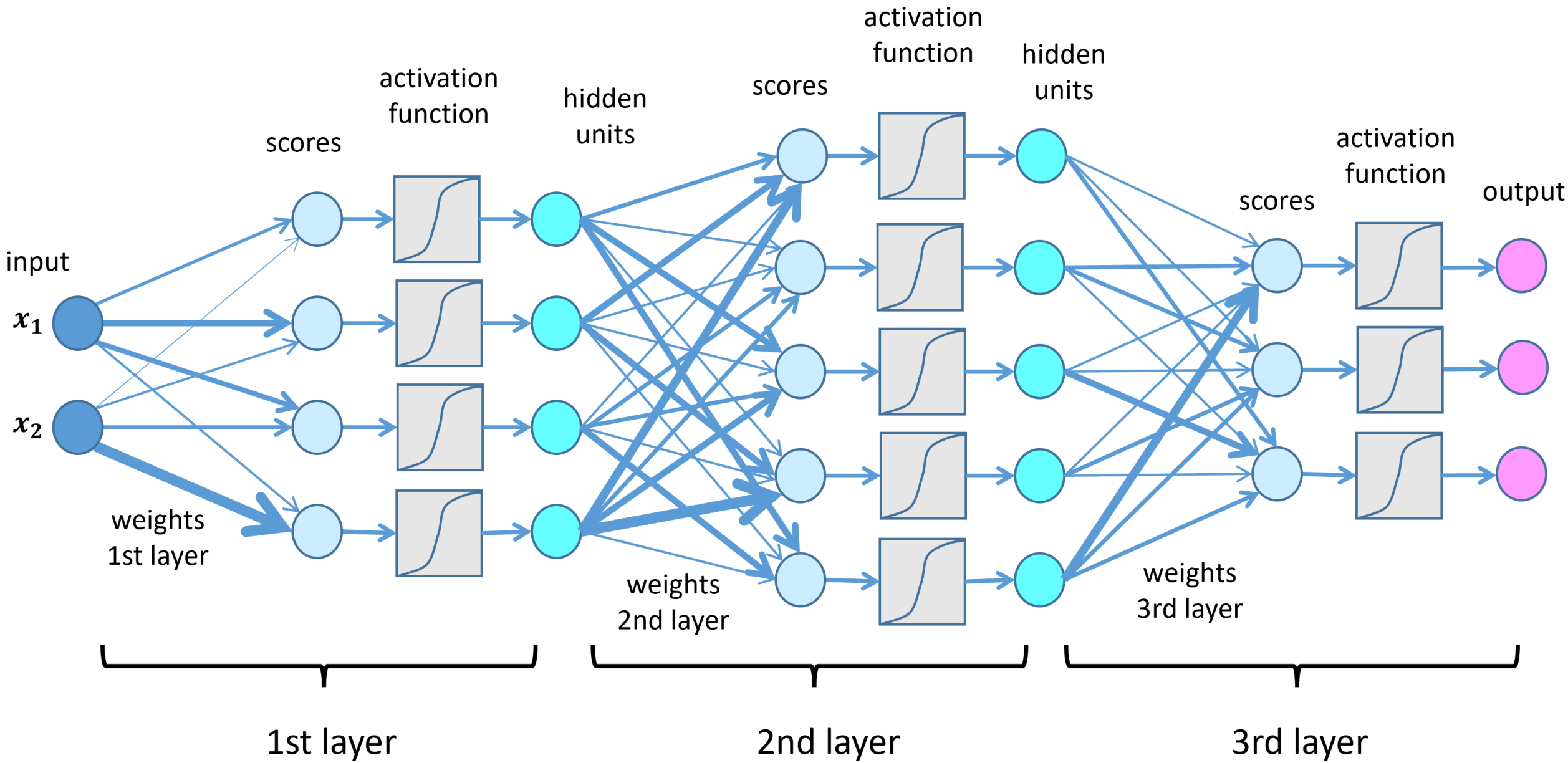
$= w_1x_1 + w_2x_2 + \ldots w_nx_n + b$

$= w^Tx+b$

$y = \frac{1}{1+e^{-(W^TX+b)}}$

Sigmoid	`1 / (1 + e^(-x))`
Tanh	`(e^x - e^-x) / (e^x + e^-x)`
ReLU	`max(0, x)`
Leaky ReLU	`x if x > 0 else 0.01x`




input
x_1
x_2
scores
activation function
hidden units
weights 1st layer
scores
activation function
hidden units
weights 2nd layer
scores
activation function
output
weights 3rd layer
1st layer
2nd layer
3rd layer

Text Generation using Keras

Why scikit-learn Can't Do Text Generation (easily)

Text generation involves:

- Understanding sequence and context (i.e., word order matters)
- Learning long-term dependencies between words
- Generating one word at a time in sequence

- These are tasks for **Recurrent Neural Networks (RNNs)** or **LSTM networks**, which are **not available in Scikit-learn**.

  You should use a **deep learning framework** such as:

  - **Keras** (simple, high-level API over TensorFlow)
  - **TensorFlow** (lower-level)
  - **PyTorch** (very popular and flexible)

Step 1:Choose a Paragraph (Training Data)

- text = "I love learning about machine learning and deep learning. Learning is fun and powerful."

Step 2: Tokenize Using Keras Tokenizer

- from keras.preprocessing.text import Tokenizer
- tokenizer = Tokenizer()
- tokenizer.fit_on_texts([text])
- print(tokenizer.word_index)

This method takes a list of texts and analyzes them to create a vocabulary. It assigns a unique integer index to each word (token) in the texts, creating a word_index dictionary that maps words to their indices. The most frequent words are typically assigned lower indices.

{'learning': 1, 'and': 2, 'i': 3, 'love': 4, 'about': 5,
'machine': 6, 'deep': 7, 'is': 8, 'fun': 9, 'powerful': 10}

Step 3: Create Input Sequences

Creating sequences from text is a key step when training models (like LSTMs or GRUs) for text generation or prediction.

```
sequences = []
tokens = tokenizer.texts_to_sequences([text])[0]
for i in range(1, len(tokens)):
    seq = tokens[:i+1]
    sequences.append(seq)

print("Sequences:")
for seq in sequences:
    print(seq)
```

[3, 4]
[3, 4, 1]
[3, 4, 1, 5]
[3, 4, 1, 5, 6]
[3, 4, 1, 5, 6, 1]
[3, 4, 1, 5, 6, 1, 2]
[3, 4, 1, 5, 6, 1, 2, 7]
...

Step 4: Pad the Sequences

- from keras.preprocessing.sequence import pad_sequences
- max_len = max([len(x) for x in sequences])
- sequences = pad_sequences(sequences, maxlen=max_len)
- print(sequences)

```
[[ 0  0  0  0  3  4]
 [ 0  0  0  3  4  1]
 [ 0  0  3  4  1  5]
 ...
]
```

Step 5: Split X and y

```
import numpy as np

sequences = np.array(sequences)
X, y = sequences[:, :-1], sequences[:, -1]
```

```
[[ 0  0  0  0  0  0  0  3  4]
 [ 0  0  0  0  0  0  3  4  1]
 [ 0  0  0  0  0  3  4  1  5]
 [ 0  0  0  0  3  4  1  5  6]
 [ 0  0  0  3  4  1  5  6  1]
 [ 0  0  3  4  1  5  6  1  2]
 [ 0  3  4  1  5  6  1  2  7]
 [ 3  4  1  5  6  1  2  7  1]
 [ 0  0  0  0  0  0  0  1  8]
 [ 0  0  0  0  0  0  1  8  9]
 [ 0  0  0  0  0  1  8  9  2]
 [ 0  0  0  0  1  8  9  2 10]]
X:
 [[0 0 0 0 0 0 0 3]
 [0 0 0 0 0 0 3 4]
 [0 0 0 0 0 3 4 1]
 [0 0 0 0 3 4 1 5]
 [0 0 0 3 4 1 5 6]
 [0 0 3 4 1 5 6 1]
 [0 3 4 1 5 6 1 2]
 [3 4 1 5 6 1 2 7]
 [0 0 0 0 0 0 0 1]
 [0 0 0 0 0 0 1 8]
 [0 0 0 0 0 1 8 9]
 [0 0 0 0 1 8 9 2]]
y:
 [ 4  1  5  6  1  2  7  1  8  9  2 10]
```

Step 7: Build a Model (LSTM Example)

```
from keras.models import Sequential
from keras.layers import Embedding, LSTM, Dense
model = Sequential()
model.add(Embedding(total_words, 10, input_length=max_seq_len-1))
model.add(LSTM(100))
model.add(Dense(total_words, activation='softmax'))
model.compile(loss='categorical_crossentropy', optimizer='adam',
metrics=['accuracy'])
```

Step 8: Train the Model

- model.fit(X, y, epochs=200, verbose=1)

The number of **iterations over the entire dataset** (you can adjust this).

The model takes X, passes them through the **Embedding → LSTM → Dense** layers.

It predicts a probability distribution over all possible words.

Compares this with the true output word (ys) using the loss function.

- Adjusts the weights to minimize the error using backpropagation and the Adam optimizer.

Step 9: Generate New Text

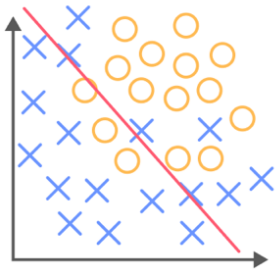
```
seed_text = "machine"
next_words = 3

for _ in range(next_words):
    token_list = tokenizer.texts_to_sequences([seed_text])[0]
    token_list = pad_sequences([token_list], maxlen=X.shape[1])
    predicted = model.predict(token_list, verbose=0)
    predicted_word = tokenizer.index_word[np.argmax(predicted)]
    seed_text += " " + predicted_word

print("Generated Text:", seed_text)
```

Generated Text: machine learning is

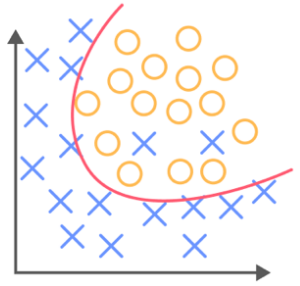
Overfitting and Underfitting in ML



- The model learns the training data *too well*, including noise and outliers.

Symptoms:

- **High accuracy on training data**
- **Poor generalization to unseen data (test/validation)**



- The model is too simple to learn the underlying patterns in the data.

Symptoms:

- **Low accuracy on both training and testing data**

Balanced and Good fit:

- Model learns the data well and generalizes to new data.
- High training and testing accuracy

- **Long Short-Term Memory**.
  It is a **type of Recurrent Neural Network (RNN)** that is specially designed to **remember long sequences** and patterns in data — especially useful in **Natural Language Processing (NLP)**, time series, and speech.

Types of Neural Networks

